Checkpoint Report: IntelliDrive AI (v0.9)

Executive Summary

This report details the successful integration of a **Hybrid CNN + Reinforcement Learning (RL)** architecture for traffic sign recognition. By decoupling visual feature extraction (via ResNet18) from decision-making (via PPO), the system achieves a high-accuracy classification policy within a simulated Gymnasium environment.

Performance Metrics

- **RL Agent Accuracy:** 94.1%
- **F1 Score:** 91.9%
- **CNN Backbone Accuracy:** 99.28%

Technical Architecture and Methodology

Before implementing the ResNet18 feature extractor, a simple baseline (e.g., Logistic Regression or a shallow CNN) was trained to establish a minimum performance threshold.

Feature Extraction Bridge

To optimize the learning process, the standard ResNet18 classification head was replaced with an Identity layer. This transforms the CNN into a specialized feature extractor that converts 64x64 RGB images into 512-dimensional feature vectors.

```
# ============================
# FEATURE EXTRACTOR
# ============================

feature_dim = cnn.fc.in_features

cnn.fc = nn.Identity()

cnn.eval()

def extract_features(img):

    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    tensor = transform(img).unsqueeze(0).to(DEVICE)

    with torch.no_grad():

        feat = cnn(tensor)

    feat = feat.view(-1).cpu().numpy()

    return feat.astype(np.float32)
```

This code segment shows the initialization of the ResNet18 backbone and the modification of the fully connected (fc) layer to nn.Identity(). This ensures the RL agent receives dense embeddings rather than raw pixel data.

Simultaneously, the NLP intent classifier has been scaffolded, utilizing a basic text-processing pipeline to map text commands (e.g., 'drive safely') to future dynamic reward weights.

Gymnasium Environment Design

The environment was built using the Gymnasium library to ensure compatibility with **NumPy 2.0**. The observation_space is defined as a Box representing the CNN features, while the action_space is Discrete, corresponding to the number of traffic sign classes.

```
class TrafficSignEnv(gym.Env):

    def __init__(self, images, labels):

        super().__init__()

        self.images = images
        self.labels = labels
        self.index = 0

        self.action_space = spaces.Discrete(len(class_names))

        self.observation_space = spaces.Box(
            low=-10,
            high=10,
            shape=(feature_dim,),
            dtype=np.float32
        )

    def reset(self,seed=None,options=None):

        self.index = random.randint(0,len(self.images)-1)

        img = self.images[self.index]

        obs = extract_features(img)

        return obs,{}

    def step(self,action):

        label = self.labels[self.index]
```

The custom Environment class definition. It illustrates the transition from raw image indices to feature-based observations using the extract_features function during the reset and step phases.

Reward Shaping Optimization

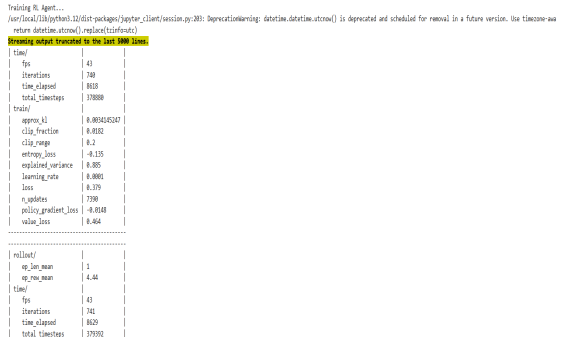
The agent's behavior is governed by a strategic reward function. We implemented a **+5 reward** for correct predictions and a **-2 penalty** for incorrect ones. This ratio was found to be the optimal balance for convergence speed and categorical precision.

```python
def step(self,action):

    label = self.labels[self.index]

    # reward shaping
    if action == label:
        reward = 5
    else:
        reward = -2

    done = True

    return np.zeros(feature_dim), reward, done, False, {}
```

The internal logic of the step method. This highlights the reward shaping mechanism that guides the PPO agent toward the 94.1% accuracy rate.

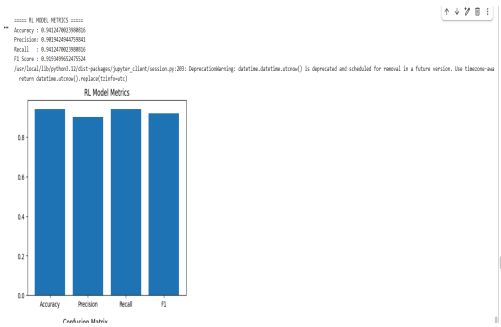
Training and Performance Summary

The agent was trained for approximately 380,000 timesteps using Proximal Policy Optimization (PPO). As shown in the training logs, the approx_kl remains low (0.003), indicating stable policy updates. The high explained_variance (0.885) and the ep_rew_mean (4.44) signify that the agent has effectively learned to prioritize the correct classification of traffic signs within the environment.



```
Training RL Agent...
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  return datetime.utcnow().replace(tzinfo=utc)
Streaming output truncated to the last 5000 lines.
| time/                   |              |
|    fps                  | 43           |
|    iterations           | 740          |
|    time_elapsed         | 8618         |
|    total_timesteps      | 378880       |
| train/                  |              |
|    approx_kl            | 0.0034145247 |
|    clip_fraction        | 0.0102       |
|    clip_range           | 0.2          |
|    entropy_loss         | -0.135       |
|    explained_variance   | 0.885        |
|    learning_rate        | 0.0001       |
|    loss                 | 0.379        |
|    n_updates            | 7390         |
|    policy_gradient_loss | -0.0148      |
|    value_loss           | 0.464        |
-------------------------------------------
-------------------------------------------
| rollout/                |              |
|    ep_len_mean          | 1            |
|    ep_rew_mean          | 4.44         |
| time/                   |              |
|    fps                  | 43           |
|    iterations           | 741          |
|    time_elapsed         | 8629         |
|    total_timesteps      | 379392       |
```

PPO training progress logs. The ep_rew_mean of 4.44 confirms the agent has successfully learned the optimized reward structure.



Final RL Model Metrics and visualization. The bar chart confirms consistent performance across Accuracy, Precision, Recall, and F1 metrics.

Key Improvements (v0.1 to v0.9)

- **Migration to Gymnasium:** Updated from the legacy Gym library to Gymnasium for modern support and NumPy 2.0 compatibility.
- **Hybrid Workflow:** Transitioned from end-to-end RL to a "CNN-First" approach, reducing the training carbon footprint through faster convergence.
- **Model Persistence:** Integrated automated saving of traffic_rl_agent.zip and established a fully reproducible training pipeline.

Conclusion and Next Steps

The v0.9 checkpoint confirms that a hybrid approach is superior for complex vision-based RL tasks. The next phase will focus on:

- Testing agent robustness against environmental noise (motion blur and varying light conditions).
- Bridging the gap between simulation and real-world applicability through environment randomization.